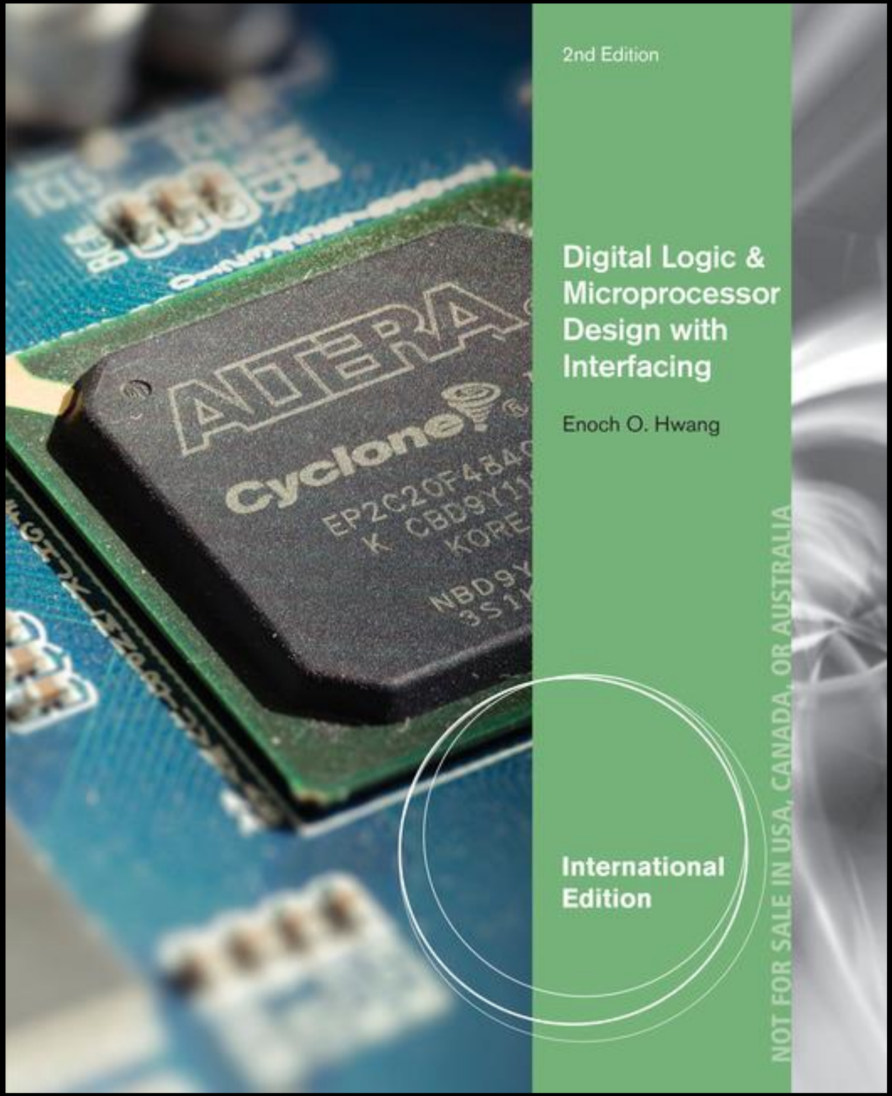


Chapter 1 Introduction to Microprocessor Design

Introduction

- All electronic devices are controlled by a microprocessor
- From simple electronic toys to complex computer systems
- Microprocessors are at the heart of smart devices

Overview

- A microprocessor is an electronic digital logic circuit implemented inside an integrated circuit (IC) chip
- At the lowest level, it can only understand whether there is electricity or no electricity
- We use 1’s and 0’s to represent this
- The question is, “How do we translate the 1’s and 0’s into a microprocessor circuit that can do something meaningful?”

Overview

- To design a microprocessor is to design its logic circuit to do whatever it is intended to do
- To implement the microprocessor is to put the logic circuit onto an IC chip

Field-Programmable Gate Array (FPGA)

- Use a Field-Programmable Gate Array (FPGA) chip to implement a digital circuit
- Large capacity – digital circuits of almost any size can be easily and quickly implemented on an FPGA
- Erasable – can be reused for different circuits

Goal

- Learn about digital logic and microprocessor design
- Understand the theory of digital logic circuits
- Be able to design and implement dedicated microcontrollers and general-purposed microprocessors
- How to interface the microprocessors with the real world

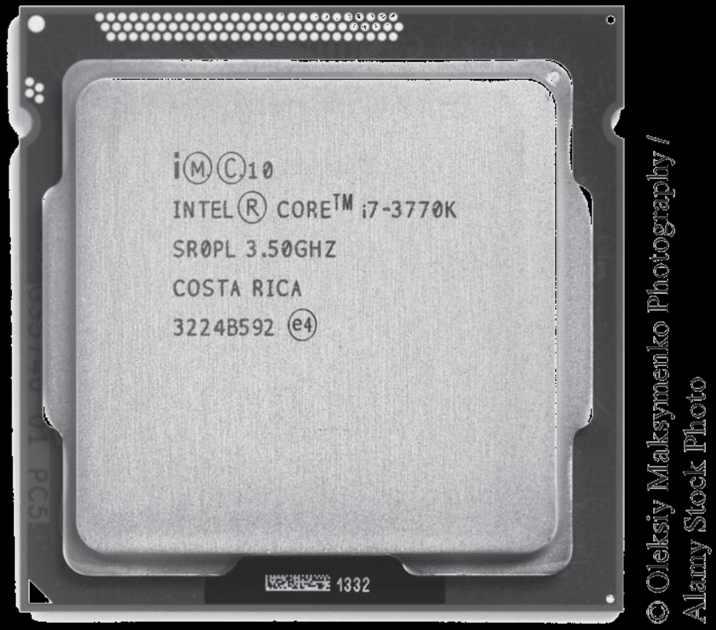
Goal

- At the end of the course, you will know how to design and implement complex digital system circuits and interface them with the real world
- The exciting part is that you will implement your very own custom digital circuit in an IC and see that it really can make lights flash, or do whatever you have designed it to do!

Types of Microprocessors

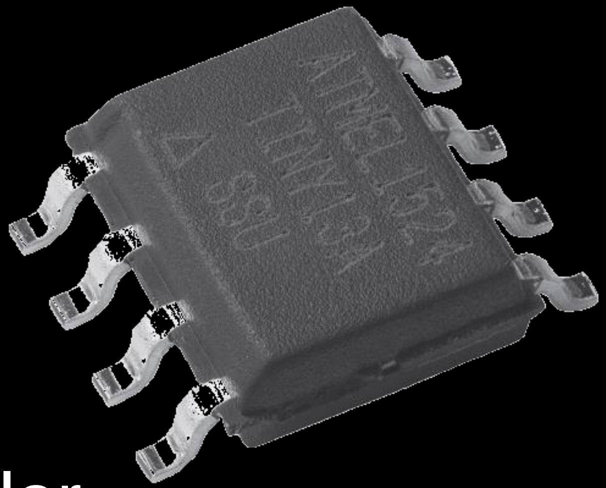
- There are two types of microprocessors:
  1. Dedicated microprocessors or microcontrollers
     - Can do only one thing
     - E.g., Traffic light controller
  2. General-purposed microprocessors
     - Can do many things by executing different programs
     - E.g., Intel Core i7 CPU can be programmed to do different things

General-Purpose Microprocessor



- Typically much more powerful in terms of processing power and speed
- Requires external components for its memory and supporting I/O peripherals

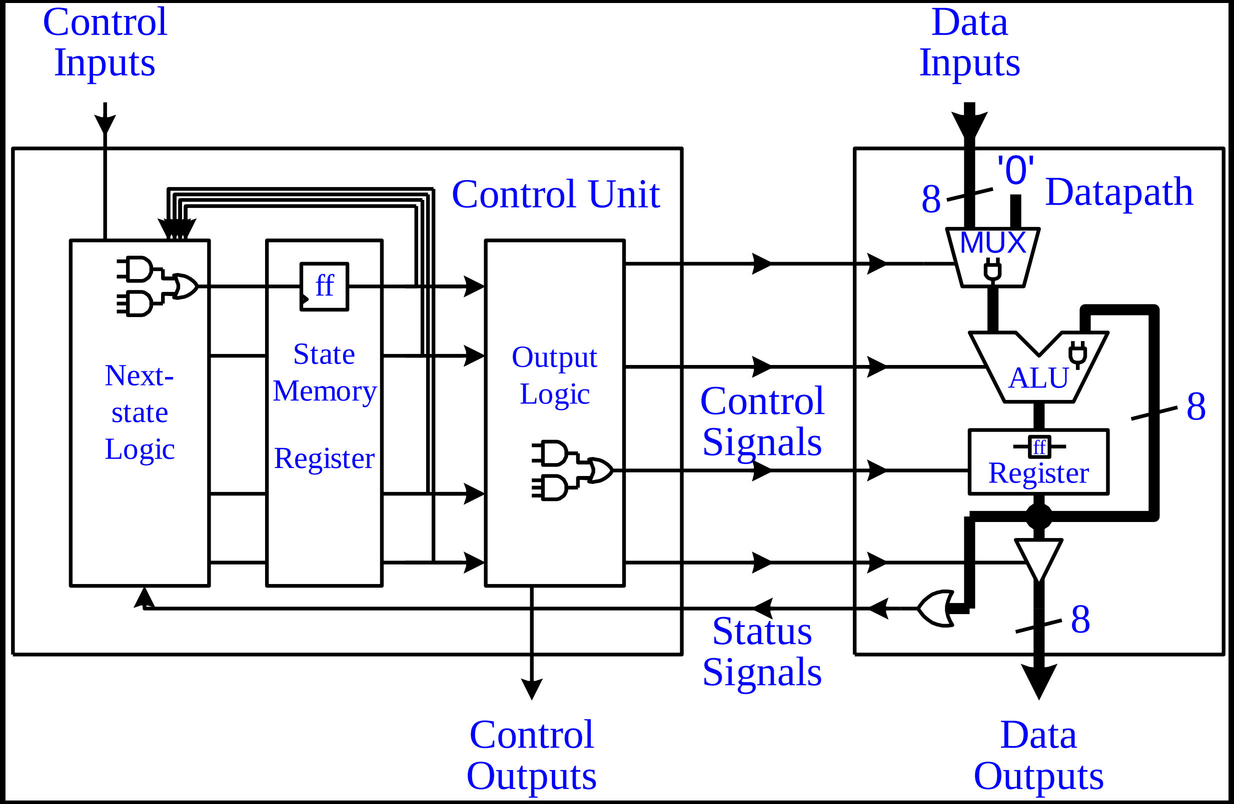
Dedicated Microprocessor



Dmitry S. Gordienko / Shutterstock.com

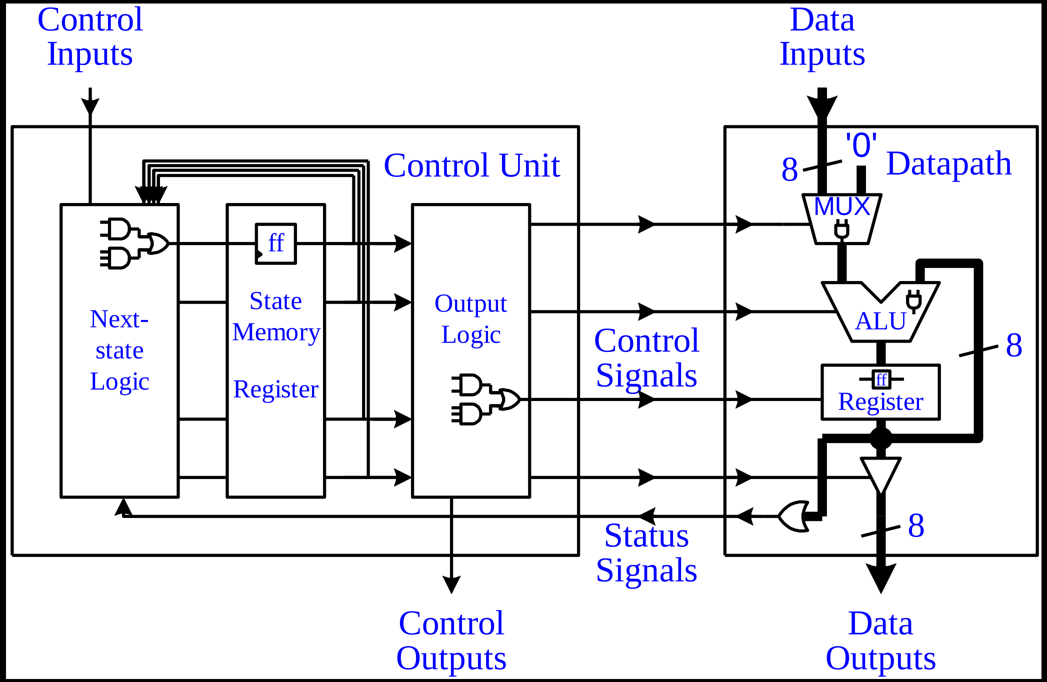
- Also known as a microcontroller
- Usually not as powerful and smaller in size
- Slower speed
- Usually has internal memory and supporting I/O peripherals
- Bigger market

Microprocessor Circuit Overview



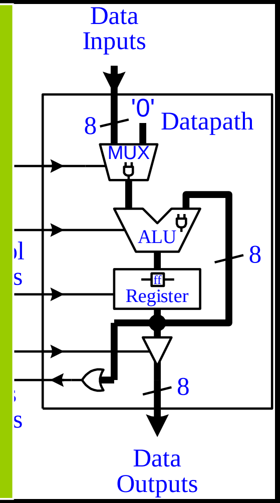
Microprocessor Circuit Overview

- Two main components:
  1. Datapath
  2. Control Unit



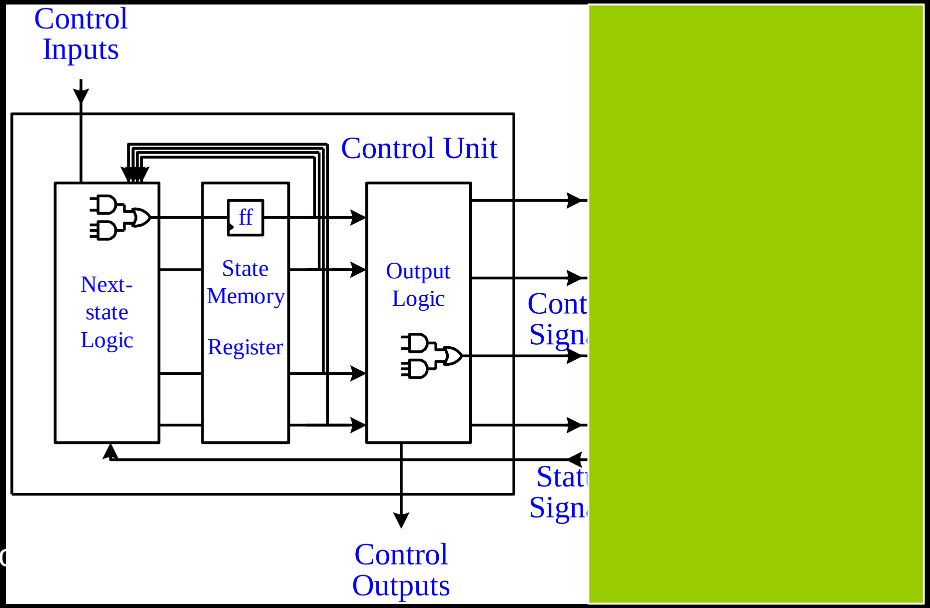
Datapath

- Responsible for the actual execution of all data operations
- Adders and ALUs for data manipulations
- Registers for temporary storage of data
- Comparators for testing data
- Buses and multiplexers for channeling data



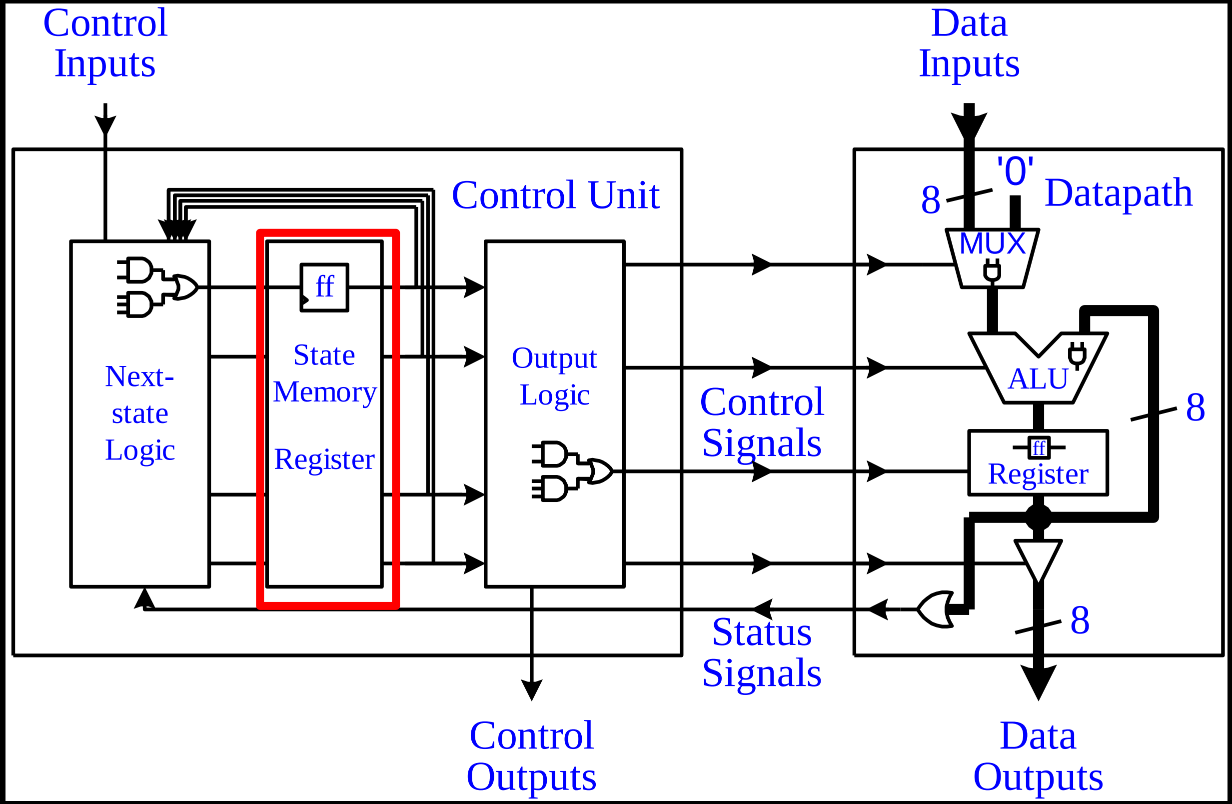
Control Unit

- Also known as the controller or the finite-state machine (FSM)
- Controls the operations of the datapath
- Controls when and which datapath operations are to be performed
- Three main parts:
    1. Next-state logic
    2. State memory
    3. Output logic



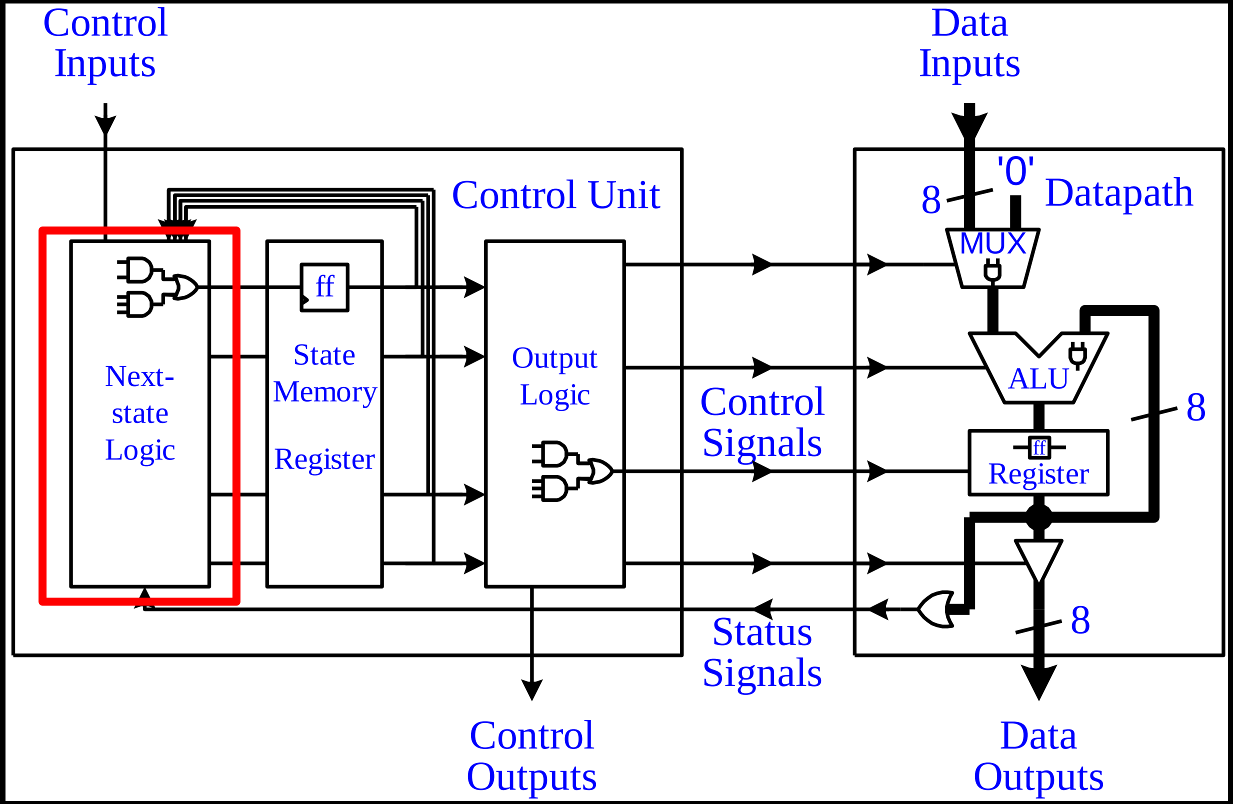
Control Unit – State Memory

- To remember the current state that the FSM is in



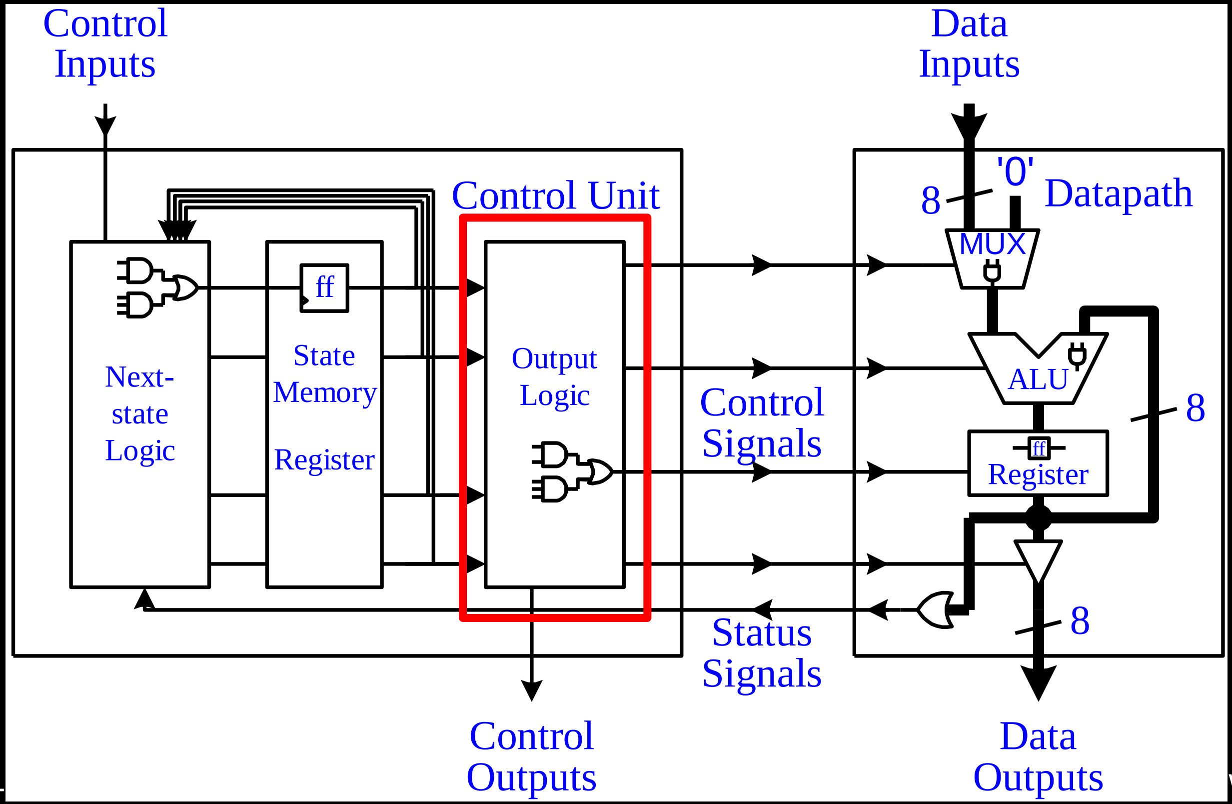
Control Unit – Next-State Logic

- To determine what the next state should be for the FSM



Control Unit – Output Logic

- To generate the control signals for controlling the datapath or external devices



Digital Logic Circuit

- All digital logic circuits are categorized as either:

    1. Combinational circuits
    2. Sequential circuits

Combinational Circuit

- The output of the circuit is dependent only on the current inputs to the circuit
- Has no memory of what has happened before
- E.g., Adder - given any two input numbers, it will produce a sum

Sequential Circuit

- The output of the circuit is dependent not only on the current inputs, but also on all of the previous inputs
- Has to remember its past history
- Must contain memory elements
- E.g., Register – can remember a value indefinitely

Logic Gates

- All digital logic circuits, whether combinational or sequential, are made up of the three basic logic gates:
  1. AND gate
  2. OR gate
  3. NOT gate
- From these three basic gates, the most powerful computer can be made

Transistors

- The fundamental building blocks for all digital logic circuits
- The logic gates are built using transistors
- Electronic binary switches that can be turned on and off
- The 1’s and 0’s that we talk about are used to represent the on and off states of a transistor

Summary

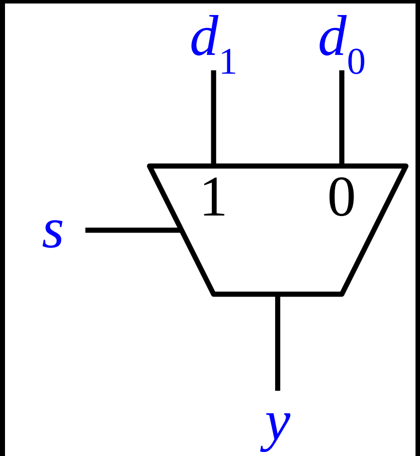
- Transistors to build logic gates
- Logic gates are connected together to form either combinational or sequential circuits
- Combinational and sequential circuits are connected together to form the control unit and the datapath
- Combining the datapath and the control unit together will produce either a dedicated or a general-purpose microprocessor

Appendix

Design Abstraction Levels

- Digital circuits can be designed at any one of several abstraction levels:
  1. Transistor level
  2. Gate level (circuit formed by gates)
  3. Dataflow level (only for combinational ckt)
  4. Register-transfer level (data transfer btw registers)
  5. Behavioral level (ckt triggered by conditions)

Example of a 2-to-1 Mux



- The 2-to-1 multiplexer has two data inputs, d_1 and d_0, and a select input, *s*.
- The select input determines which data from the two data inputs will pass to the output, *y*

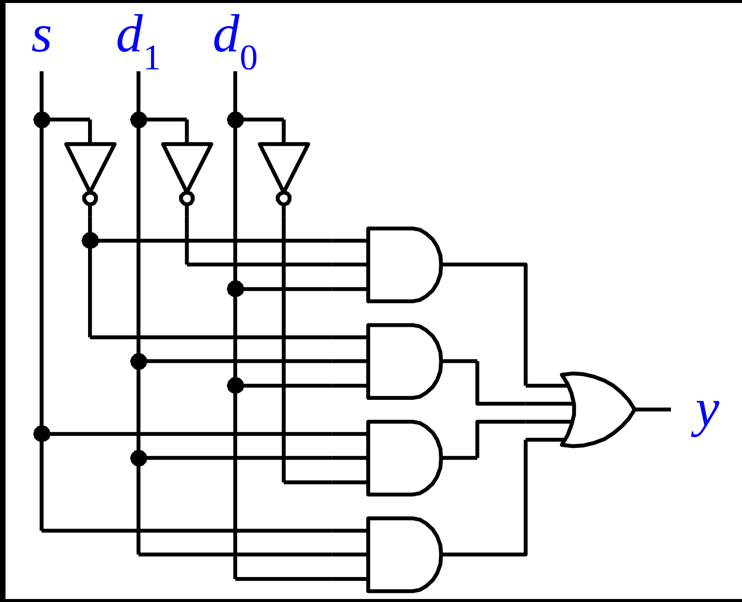
Behavioral Level

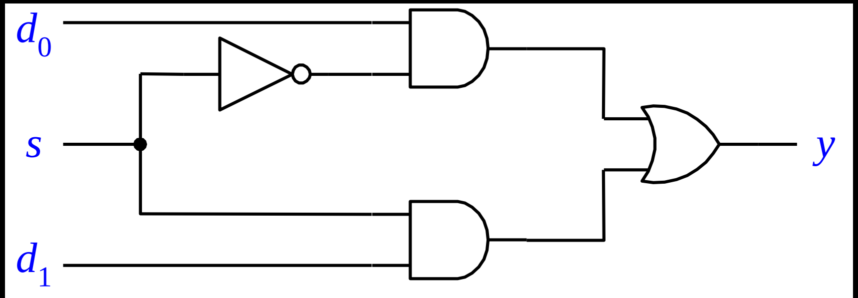
- Verilog code

```
module multiplexer (
    input s, d0, d1,
    output reg y
);
    always @ (s or d0 or d1) begin
        if (s == 0) begin
            y = d0;
        end else begin
            y = d1;
        end
    end
endmodule
```

Gate Level

- Schematic diagram of 2-to-1 mux





Gate Level

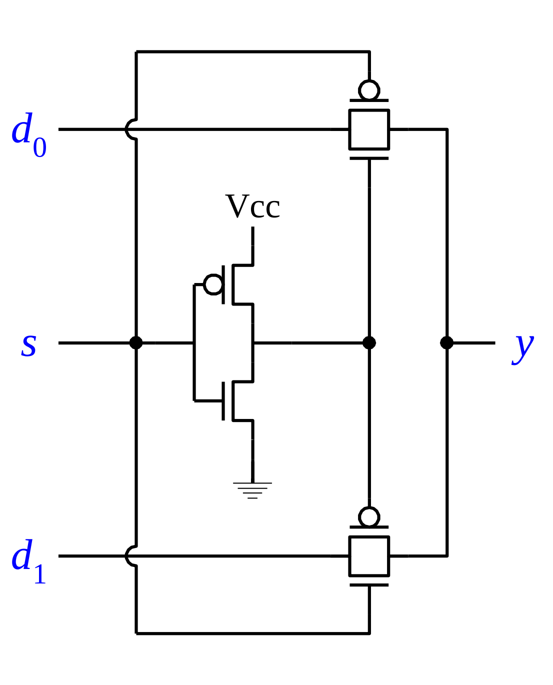
- Truth table and Boolean equation

s	d_1	d_0	y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$$y = s'\,d1'\,d0 + s'\,d1\,d0 + s\,d1\,d0' + s\,d1\,d0$$
$$= s'\,d0 + s\,d1$$

Transistor Level

- Using NMOS and PMOS transistors



Hardware Description Language

- Two popular HDLs used today are VHDL and Verilog

Verilog

- Dataflow level Verilog code of 2-to-1 mux

```
module multiplexer (
    input s, d0, d1,
    output y
);

    assign y = ((~s) & d0) | (s & d1);
endmodule
```

VHDL

- Dataflow level VHDL code of 2-to-1 mux

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY multiplexer IS PORT(
    s, d0, d1: IN STD_LOGIC;
    y: OUT STD_LOGIC);
END multiplexer;

ARCHITECTURE Dataflow OF multiplexer IS
BEGIN
    y <= ((NOT s) AND d0) OR (s AND d1);
END Dataflow;
```

Synthesis

- A synthesizer is used to translate the HDL program into the circuit **netlist**
- A netlist is a description of how a circuit actually is realized or connected using basic gates
- This translation process from an HDL description of a circuit to its netlist is referred to as **synthesis**

Implementation

- The netlist from the output of the synthesizer can be used directly to implement the actual circuit in a Field Programmable Gate Array (FPGA) integrated circuit (IC) chip